

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1401

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO1: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

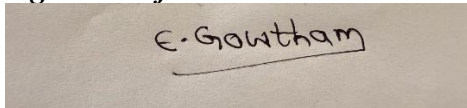
Duration :60 Mins
On Class
Total Marks:50

Code of Conduct:

I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink. The signature is written in a cursive style with a horizontal line underneath the name.

Annexure A

CASE STUDY

Q1. A compiler developer is designing a semantic analyzer to process arithmetic expressions and generate an internal representation using syntax trees. The analyzer must also define appropriate syntax-directed definitions (SDD) to compute and represent expressions correctly during parsing.

The developer is given the following expression:

$a + b * c - d$

The system must ensure:

- Correct operator precedence ($*$ $>$ $+$ $>$ $-$)
- Proper left associativity of operators
- Accurate construction of syntax tree
- Definition of semantic rules for evaluation

Tasks:

- (i) Construct the syntax tree for the given expression considering operator precedence and associativity.
- (ii) Write the syntax-directed definition (SDD) for the grammar used to generate such expressions.
- (iii) Show how attributes are evaluated for the given expression.
- (iv) Explain how the syntax tree represents the correct order of operations. (10 Marks)

Q2. A compiler developer is implementing a semantic analyzer for arithmetic expressions using S-attributed syntax-directed definitions (SDD). The system is designed to evaluate expressions and simultaneously generate intermediate code using a stack-based parser (bottom-up evaluation).

The following grammar is used to define arithmetic expressions:

$$S \rightarrow E \text{ n}$$
$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow (E) \mid \text{digit}$$

The input string to be processed is:

8 + 2 * 4 n

The compiler must:

- Evaluate the expression using bottom-up parsing
- Apply S-attributed definitions
- Generate corresponding intermediate code (translator output)

Tasks:

(i) Construct the S-attributed syntax-directed definition (SDD) for the given grammar, specifying semantic rules for each production.

(ii) Develop a code fragment (translator) to evaluate expressions using the defined SDD.

(iii) Use a parser stack to demonstrate the step-by-step evaluation of the input string:

8 + 2 * 4 n

- Show stack contents
- Show intermediate computations

(iv) Construct the syntax tree for the given expression and annotate it with computed values.

(v) Explain how bottom-up evaluation using S-attributed definitions ensures correct operator precedence and associativity. (10 Marks)

Q3. A compiler developer is designing a semantic analyzer using syntax-directed definitions (SDD). The analyzer must determine whether given attribute rules follow valid evaluation strategies such as S-attributed or L-attributed definitions, and whether they can be evaluated without conflicts.

Consider the production:

$$A \rightarrow B C D$$

Each non-terminal (A, B, C, D) has two attributes:

- $s \rightarrow$ synthesized attribute
- $i \rightarrow$ inherited attribute

The developer is given the following sets of semantic rules:

Set (i):

$$A.s = B.i + C.i$$

Set (ii):

$$A.s = B.i + C.s$$
$$D.i = A.i + B.s$$

Set (iii):

$$A.s = B.s + D.s$$

Set (iv):

$$A.s = D.i$$
$$B.i = A.s + D.s$$
$$C.i = B.s$$
$$D.i = B.i + C.i$$

Tasks:

- (i) For each set of rules, determine whether they are consistent with an S-attributed definition.
- (ii) Determine whether the rules satisfy the conditions of an L-attributed definition.
- (iii) Analyze whether the rules allow a valid evaluation order (i.e., no circular dependency).
- (iv) For each set, justify your answer by explaining the attribute dependency relationships.
- (v) Identify any circular dependencies and explain why they prevent evaluation.(10 Marks)

Q4. A compiler developer is designing a semantic analyzer for a programming language that supports variable declarations. During semantic analysis, the compiler must propagate type information from a type specifier to a list of identifiers and update the symbol table accordingly.

The compiler uses the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int} \mid \text{real}$

$L \rightarrow L, \text{id} \mid \text{id}$

The input declaration is:

real p, q, r

The semantic analyzer must perform bottom-up evaluation of inherited attributes while parsing the declaration and ensure that the type information is correctly assigned to all identifiers.

Tasks

- (i). Construct a suitable Syntax-Directed Definition (SDD) for the given grammar to propagate type information from T to all identifiers in L.
- (ii). Identify the inherited and synthesized attributes used in the grammar and explain their roles.
- (iii). Demonstrate the bottom-up evaluation process for the input string:
real p, q, r
showing how attribute values are computed during reductions.

(iv). Construct the annotated parse tree and clearly indicate the attribute values associated with each node.

(v). Illustrate the stack implementation of bottom-up parsing, showing:

Stack contents

Remaining input

Parser action (Shift/Reduce)

Attribute evaluation performed at each reduction (10 Marks)

Q5. A compiler developer is designing a semantic analyzer that performs top-down translation of arithmetic expressions using Syntax-Directed Definitions (SDD). The analyzer must evaluate arithmetic expressions and construct an annotated parse tree during parsing.

The following grammar is used:

$S \rightarrow E n$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{digit}$

The input expression is:

$3 * 4 + 6 n$

The developer must ensure:

Proper top-down evaluation using L-attributed definitions

Construction of an annotated parse tree

Correct computation of expression values

Proper handling of operator precedence and associativity

Tasks

(i). Construct a suitable Syntax-Directed Definition (SDD) for the given grammar using L-attributed definitions.

(ii). Show how inherited and synthesized attributes are propagated during top-down parsing.

(iii). Perform the top-down translation for the input string:

$3 * 4 + 6 n$ and compute the final expression value.

(iv). Construct the annotated parse tree, clearly indicating attribute values at each node. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Syntax Tree and Syntax-Directed Definition

Given expression:

$$a + b * c - d$$

Operator precedence:

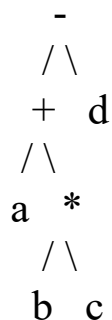
$\cdot | \cdot | \cdot$

All operators are left associative.

Therefore, the expression is evaluated as:

$$(a + (b * c)) - d$$

(i) Syntax Tree



The root node is $-$. Its left child is $+$, and the right child is d . The $+$ node has a as its left child and $*$ as its right child. The $*$ node has b and c as its children.

(ii) Syntax-Directed Definition

Grammar:

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{id}$

Use the synthesized attribute val.

Semantic rules:

$E \rightarrow E1 + T$

$E.val = E1.val + T.val$

$E \rightarrow E1 - T$

$E.val = E1.val - T.val$

$E \rightarrow T$

$E.val = T.val$

$T \rightarrow T1 * F$

$T.val = T1.val * F.val$

$T \rightarrow T1 / F$

$T.val = T1.val / F.val$

$T \rightarrow F$

$T.val = F.val$

$F \rightarrow (E)$

$F.val = E.val$

$F \rightarrow id$

$F.val = id.lexval$

(iii) Attribute Evaluation

For $a + b * c - d$:

First evaluate:

$b * c$

Therefore:

$T.val = b * c$

Next evaluate:

$a + (b * c)$

Therefore:

$E.val = a + (b * c)$

Finally evaluate: $(a + (b * c)) - d$

Therefore: $E.val = (a + (b * c)) - d$

(iv) Explanation

The syntax tree represents the correct order of operations because multiplication has higher precedence than addition and subtraction. Therefore:

1. $b * c$ is evaluated first.
2. $a + (b * c)$ is evaluated next.
3. $(a + (b * c)) - d$ is evaluated last.

Hence, the syntax tree correctly represents:

$(a + (b * c)) - d$

This also shows left associativity of $+$ and $-$.

Q2. S-Attributed SDD and Bottom-Up Evaluation

Given grammar:

$S \rightarrow E n$

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{digit}$

Input:

$8 + 2 * 4 n$

The result is: $8 + (2 * 4) = 16$

(i) S-Attributed SDD

Use synthesized attribute val.

Semantic rules:

$S \rightarrow E n$

$S.val = E.val$

$E \rightarrow E1 + T$

$E.val = E1.val + T.val$

$E \rightarrow E1 - T$

$$E.val = E1.val - T.val$$

$$E \rightarrow T$$

$$E.val = T.val$$

$$T \rightarrow T1 * F$$

$$T.val = T1.val * F.val$$

$$T \rightarrow T1 / F$$

$$T.val = T1.val / F.val$$

$$T \rightarrow F$$

$$T.val = F.val$$

$$F \rightarrow (E)$$

$$F.val = E.val$$

$$F \rightarrow \text{digit}$$

$$F.val = \text{digit.lexval}$$

Since only synthesized attributes are used, this is an S-attributed SDD.

(ii) Translator Code

A simple translator can be written as:

evaluate(node):

if node is digit:
 return node.lexval

if node is $E \rightarrow E1 + T$:
 return evaluate(E1) + evaluate(T)

if node is $E \rightarrow E1 - T$:
 return evaluate(E1) - evaluate(T)

if node is $T \rightarrow T1 * F$:
 return evaluate(T1) * evaluate(F)

if node is $T \rightarrow T1 / F$:
 return evaluate($T1$) / evaluate(F)

if node is $E \rightarrow T$:
 return evaluate(T)

if node is $T \rightarrow F$:
 return evaluate(F)

Intermediate code:

$t1 = 2 * 4$

$t2 = 8 + t1$

Therefore:

$t1 = 8$

$t2 = 16$

Final result = 16.

(iii) Parser Stack Evaluation

Input:

$8 + 2 * 4 n$

Step	Stack	Input	Action
1	0	$8 + 2 * 4 n$	Shift 8
2	0 8	$+ 2 * 4 n$	Reduce digit $\rightarrow F$
3	0 F	$+ 2 * 4 n$	Reduce $F \rightarrow T$
4	0 T	$+ 2 * 4 n$	Reduce $T \rightarrow E$
5	0 E	$+ 2 * 4 n$	Shift +
6	0 E +	$2 * 4 n$	Shift 2
7	0 E + 2	$* 4 n$	Reduce digit $\rightarrow F$
8	0 E + F	$* 4 n$	Reduce $F \rightarrow T$
9	0 E + T	$* 4 n$	Shift *
10	0 E + T *	4 n	Shift 4
11	0 E + T * 4	n	Reduce digit $\rightarrow F$
12	0 E + T * F	n	Reduce $T \rightarrow T * F$
13	0 E + T	n	Compute $2 * 4 = 8$

Step	Stack	Input	Action
14	0 E	n	Reduce $E \rightarrow E + T$
15	0 E	n	Compute $8 + 8 = 16$
16	0 E n	empty	Shift n
17	0 S	empty	Reduce $S \rightarrow E n$

Final Value: 16

(iv) Syntax Tree

$$\begin{array}{c}
 + \\
 /\backslash \\
 8 \quad * \\
 /\backslash \\
 2 \quad 4
 \end{array}$$

Annotated values:

$8 \rightarrow 8$

$2 \rightarrow 2$

$4 \rightarrow 4$

$2 * 4 = 8$

$8 + 8 = 16$

Therefore: Expression value = 16

(v) Explanation

Bottom-up evaluation gives correct operator precedence because the grammar places multiplication at the T level and addition at the E level.

Therefore, the parser reduces:

$2 * 4$

before:

$$8 + 8$$

Hence:

$$8 + 2 * 4 = 16$$

The left-recursive production:

$$E \rightarrow E + T$$

also gives left associativity.

Q3. Attribute Dependency Analysis

Given:

$$A \rightarrow B C D$$

Each non-terminal has:

s = synthesized attribute

i = inherited attribute

(i) Set (i)

Given:

$$A.s = B.i + C.i$$

S-attributed?

No.

Reason: B.i and C.i are inherited attributes. S-attributed definitions allow only synthesized attributes.

L-attributed?

Yes, because the rule defines a synthesized attribute of A.

Circular dependency?

No circular dependency is given.

Answer:

S-attributed: No

L-attributed: Yes

Circular dependency: No

(ii) Set (ii)

Given:

$$A.s = B.i + C.s$$

$$D.i = A.i + B.s$$

S-attributed?

No, because inherited attributes are present.

L-attributed?

The rule: $B.i = A.s + D.s$

defines an inherited attribute of B using D.s, but D occurs to the right of B. This violates the L-attributed condition.

Circular dependency?

Yes.

The dependency is:

$D.i \rightarrow A.s \rightarrow B.i \rightarrow D.i$

This forms a cycle.

Therefore, there is no valid evaluation order.

Answer:

S-attributed: No

L-attributed: No

Circular dependency: Yes

Valid evaluation order: No

Summary:

Set S-attributed L-attributed Circular

(i) No	Yes	No
(ii) No	Yes	No
(iii) Yes	Yes	No
(iv) No	No	Yes

Q4. Type Propagation

Given grammar:

$D \rightarrow T L$

$T \rightarrow \text{int} \mid \text{real}$

$L \rightarrow L, \text{id} \mid \text{id}$

Input:

real p, q, r

(i) Syntax-Directed Definition

Use:

T.type

L.in

id.type

Semantic rules:

$D \rightarrow T L$

L.in = T.type

$T \rightarrow \text{int}$
 $T.\text{type} = \text{integer}$

$T \rightarrow \text{real}$
 $T.\text{type} = \text{real}$

$L \rightarrow L1, \text{id}$
 $L1.\text{in} = L.\text{in}$
 $\text{id}.\text{type} = L.\text{in}$

$L \rightarrow \text{id}$
 $\text{id}.\text{type} = L.\text{in}$

For:

real p, q, r

we get:

$T.\text{type} = \text{real}$

Then:

$L.\text{in} = \text{real}$

Therefore:

$p.\text{type} = \text{real}$

$q.\text{type} = \text{real}$

$r.\text{type} = \text{real}$

(ii) Attributes Used

Synthesized attribute:

$T.\text{type}$

It obtains the type from:

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

Inherited attribute:

$L.\text{in}$

It carries the type information from T to the identifier list.

Identifier attribute:

$\text{id}.\text{type}$

stores the type of each identifier.

Final values:

p.type = real

q.type = real

r.type = real

(iii) Bottom-Up Evaluation

Input:

real p, q, r

First:

real

is reduced using:

$T \rightarrow \text{real}$

Therefore:

T.type = real

Then:

L.in = T.type

So:

L.in = real

For p:

p.type = real

For q:

q.type = real

For r:

r.type = real

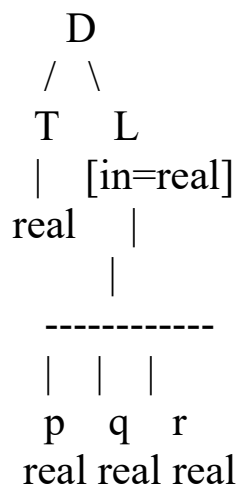
Final result:

$p \rightarrow \text{real}$

$q \rightarrow \text{real}$

$r \rightarrow \text{real}$

(iv) Annotated Parse Tree



Therefore, all three identifiers have type real.

(v) Stack Implementation

Step	Stack	Remaining Input	Action
1	\$	real p , q , r	Shift real
2	\$ real	p , q , r	Reduce $T \rightarrow \text{real}$
3	\$ T	p , q , r	Shift p
4	\$ T p	, q , r	Reduce $L \rightarrow \text{id}$
5	\$ T L	, q , r	Shift ,
6	\$ T L ,	q , r	Shift q
7	\$ T L , q	, r	Reduce id
8	\$ T L , r		Reduce $L \rightarrow L , \text{id}$
9	\$ T L , r		Shift r
10	\$ T L , r	empty	Reduce id
11	\$ T L	empty	Reduce $L \rightarrow L , \text{id}$
12	\$ D	empty	Reduce $D \rightarrow T L$

Attribute evaluations:

$T.\text{type} = \text{real}$

$L.\text{in} = \text{real}$

$p.\text{type} = \text{real}$

$q.\text{type} = \text{real}$

$r.\text{type} = \text{real}$

Final symbol table:

Identifier	Type
------------	------

p	real
---	------

q	real
---	------

r	real
---	------

Q5. L-Attributed Top-Down Translation

Given grammar:

$S \rightarrow E n$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{digit}$

Input:

$3 * 4 + 6 \text{ n}$

The expression is:

$3 * 4 + 6$

Therefore:

$3 * 4 = 12$

$12 + 6 = 18$

Final answer:

18

Important: The given grammar is left recursive. For top-down parsing, left recursion must be removed.

Use the equivalent grammar:

$S \rightarrow E \text{ n}$

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow (E) \mid \text{digit}$

(i) L-Attributed SDD

Semantic rules:

$S \rightarrow E \text{ n}$

$S.\text{val} = E.\text{val}$

$E \rightarrow T E'$

$E'.\text{inh} = T.\text{val}$

$E.\text{val} = E'.\text{syn}$

$E' \rightarrow + T E'1$

$E'1.\text{inh} = E'.\text{inh} + T.\text{val}$

$E'.\text{syn} = E'1.\text{syn}$

$E' \rightarrow \varepsilon$

$E'.\text{syn} = E'.\text{inh}$

$T \rightarrow F T'$

$T'.\text{inh} = F.\text{val}$

$$T.val = T'.syn$$

$$T' \rightarrow * F T'1$$

$$T'1.inh = T'.inh * F.val$$

$$T'.syn = T'1.syn$$

$$T' \rightarrow \varepsilon$$

$$T'.syn = T'.inh$$

$$F \rightarrow (E)$$

$$F.val = E.val$$

$$F \rightarrow \text{digit}$$

$$F.val = \text{digit.lexval}$$

(ii) Attribute Propagation

For:

$$3 * 4 + 6$$

First:

$$F.val = 3$$

Then:

$$T'.inh = 3$$

Read 4:

$$F.val = 4$$

Multiply:

$$\begin{aligned} T'1.inh &= 3 * 4 \\ &= 12 \end{aligned}$$

Therefore:

$$T.val = 12$$

Now addition:

$$E'.inh = 12$$

Read 6:

$$T.val = 6$$

Then:

$$\begin{aligned} E'1.inh &= 12 + 6 \\ &= 18 \end{aligned}$$

Therefore:

$$E.val = 18$$

Finally:

$S.val = 18$

(iii) Top-Down Translation

Step 1:

$S \rightarrow E n$

Step 2:

$E \rightarrow T E'$

Step 3:

$T \rightarrow F T'$

Step 4:

$F \rightarrow \text{digit}$

Read 3:

$F.val = 3$

Step 5:

Read *.

$T' \rightarrow * F T'$

Read 4:

$F.val = 4$

Calculate:

$3 * 4 = 12$

Therefore:

$T.val = 12$

Step 6:

Read +.

$E' \rightarrow + T E'$

Read 6:

$T.val = 6$

Calculate:

$12 + 6 = 18$

Therefore:

$E.val = 18$

Final:

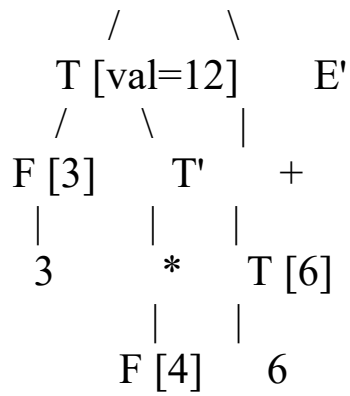
$S.val = 18$

Hence:

Final expression value = 18

(iv) Annotated Parse Tree

$E [val=18]$



Attribute evaluation:

$F.val = 3$

$F.val = 4$

$T'.inh = 3$

$T'1.inh = 3 * 4 = 12$

$T.val = 12$

$E'.inh = 12$

$F.val = 6$

$T.val = 6$

$E'1.inh = 12 + 6 = 18$

$E.val = 18$

Therefore:

$3 * 4 + 6 = 18$